

1 Solutions of Problems in Exercise Session 5

Exercise 1. Choose the correct algorithm (REINFORCE/DDPG/TRPO/A2C) for each column title and fill in the missing entries:

Solution. We emphasize that there is no single correct solutions to this exercise. The following table should be understood as a guideline:

Algorithm	TRPO	A2C	REINFORCE	DDPG
On/Off-Policy	On-policy	On-policy	On-policy	Off-policy
Action Space	Discrete/ Continuous	Discrete/ Continuous	Discrete/ Continuous	Continuous
Sample Efficiency	Moderate	Moderate	Low	High
Exploration	Implicit	Implicit	Implicit	Additive noise on actions
Stability	High	Moderate	Low (high variance)	Low
Biggest disadvantage	Computationally expensive	faster: A3C, exploration: SAC, simpler/efficiency: PPO	Requires full episodes	Only deterministic policies

Here are a few key points from the table:

On/off-policy: DDPG can be both an on-policy and off-policy algorithm (see slide 28 in Lecture 7). All off-policy algorithms can be used on-policy, but not the other way around!

Action space: All algorithms except DDPG can accept both discrete and continuous action spaces (with minor modifications). DDPG is specifically designed for continuous actions (in fact, it is sometimes referred as "DQN for continuous actions").

Sample Efficiency: REINFORCE uses samples from an episode only once, updating parameters after that episode ends. In contrast, DDPG, as an off-policy algorithm, reuses samples from the replay buffer multiple times.

Exploration: In TRPO, A2C and REINFORCE, exploration is implicit because it relies on the stochasticity of actions - recall that all three methods optimize over stochastic policies.

Stability: REINFORCE has low stability, especially without variance reduction techniques (see Exercise Session 4). Similarly, DDPG is notoriously unstable and highly sensitive to hyperparameter choices.

Biggest disadvantage: TRPO is computationally expensive due to its approximation of the natural policy gradient (requires matrix inversions). A2C, while foundational, has been significantly improved by algorithms such as A3C, SAC and PPO.

Exercise 2. To develop a better understanding of soft-value functions, solve:

1. Show that under a deterministic policy π , $V_{\mathcal{H}}^{\pi}$ reduces to standard values V^{π} .
2. Let $\pi(\cdot|s)$ be ε -greedy policy. If $r(s, a) = R = \text{const}$, identify an optimal ε that maximizes $V_{\mathcal{H}}^{\pi}(s)$.
3. Derive a reward function $r_{\mathcal{H}}^{\pi}$ such that $V_{\mathcal{H}}^{\pi}$ and $Q_{\mathcal{H}}^{\pi}$ satisfy the standard equations of V^{π} and Q^{π} .

Solution. 1. For a deterministic policy $\pi(\cdot|s)$, we have $\pi(\bar{a}|s) = 1$ for a specific action $\bar{a}$, and $= 0$ for all other actions. Thus, the entropy of a deterministic policy is:

$$\mathcal{H}(\pi(\cdot|s)) = \mathbf{E}_{a \sim \pi(\cdot|s)} \log \left(\frac{1}{\pi(a|s)} \right) = \log \left(\frac{1}{\pi(\bar{a}|s)} \right) = \log 1 = 0$$

Therefore, the equations for $V_{\mathcal{H}}^{\pi}(s)$ and $Q_{\mathcal{H}}^{\pi}(s, a)$ simplify to:

$$\begin{aligned} V_{\mathcal{H}}^{\pi}(s) &= \mathbb{E}_{a \sim \pi(\cdot|s)} [Q_{\mathcal{H}}^{\pi}(s, a)] \\ Q_{\mathcal{H}}^{\pi}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} V_{\mathcal{H}}^{\pi}(s'), \end{aligned}$$

which are exactly the standard equations for V^{π} and Q^{π} .

2. Since all rewards are equal, we only need to maximize $V_{\mathcal{H}}^{\pi}$ by increasing the entropy $\mathcal{H}(\pi)$. For an ε -greedy policy:

$$\pi(a|s) = \begin{cases} 1 - \varepsilon, & \text{if } a = \arg\max_b Q_{\mathcal{H}}^{\pi}(s, b), \\ \frac{\varepsilon}{A-1}, & \text{otherwise} \end{cases}$$

the entropy of the policy is given by:

$$\mathcal{H}(\pi(\cdot|s)) = \sum_a \pi(a|s) \log \left(\frac{1}{\pi(a|s)} \right) = (1 - \varepsilon) \log \frac{1}{1 - \varepsilon} + \varepsilon \log \frac{A-1}{\varepsilon}$$

Taking the derivative with respect to ε , we find that the optimal ε is $\varepsilon = \frac{A-1}{A}$. This corresponds to a uniform policy $\pi(\cdot|s)$, as expected, since uniformity maximizes entropy.

3. Substituting the expression for $V_{\mathcal{H}}^{\pi}(s)$ into the equation for $Q_{\mathcal{H}}^{\pi}$, we get:

$$\begin{aligned} Q_{\mathcal{H}}^{\pi}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} V_{\mathcal{H}}^{\pi}(s') \\ &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\mathbb{E}_{a' \sim \pi(\cdot|s')} [Q_{\mathcal{H}}^{\pi}(s', a')] + \mathcal{H}(\pi(\cdot|s')) \right] \end{aligned}$$

Comparing this with the standard equation for $Q^{\pi}(s, a)$:

$$\begin{aligned} Q^{\pi}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} V^{\pi}(s') \\ &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\mathbb{E}_{a' \sim \pi(\cdot|s')} [Q^{\pi}(s', a')] \right], \end{aligned} \tag{4}$$

we can define a modified reward function:

$$r_{\mathcal{H}}^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \mathcal{H}(\pi(\cdot|s'))$$

Using this reward function, $Q_{\mathcal{H}}^{\pi}$ satisfies the standard equation for $Q^{\pi}(s, a)$, i.e. (4), with $r_{\mathcal{H}}^{\pi}(s, a)$ instead of $r(s, a)$.

Exercise 3. Show that the policy π_{new} improves upon π_{old} by demonstrating:

$$Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) \leq Q_{\mathcal{H}}^{\pi_{\text{new}}}(s, a), \quad \forall s, a.$$

Solution. By the definition of π_{new} , for any policy $\pi \in \Pi$, we have for all states s :

$$D_{\text{KL}} \left(\pi_{\text{new}}(\cdot|s) \left\| \frac{\exp(Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, \cdot))}{Z^{\pi_{\text{old}}}(s)} \right\| \right) \leq D_{\text{KL}} \left(\pi(\cdot|s) \left\| \frac{\exp(Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, \cdot))}{Z^{\pi_{\text{old}}}(s)} \right\| \right) \quad (5)$$

Expanding these terms, we have:

$$D_{\text{KL}} \left(\pi_{\text{new}}(\cdot|s) \left\| \frac{\exp(Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, \cdot))}{Z^{\pi_{\text{old}}}(s)} \right\| \right) = \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} \left[\log \frac{\pi_{\text{new}}(a|s)}{\frac{\exp(Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a))}{Z^{\pi_{\text{old}}}}} \right],$$

and can be further simplified to:

$$= \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} \log \pi_{\text{new}}(a|s) - \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) + \underbrace{\mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} \log Z^{\pi_{\text{old}}}}_{\log Z^{\pi_{\text{old}}}},$$

where we used properties of logarithms, and the fact that $\log Z^{\pi_{\text{old}}}$ does not depend on a . Using this result, inequality (5) becomes for $\pi = \pi_{\text{old}}$:

$$\begin{aligned} & \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} \log \pi_{\text{new}}(a|s) - \mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) \\ & \leq \mathbb{E}_{a \sim \pi_{\text{old}}(\cdot|s)} \log \pi_{\text{old}}(a|s) - \mathbb{E}_{a \sim \pi_{\text{old}}(\cdot|s)} Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) \end{aligned}$$

From the definition of (2):

$$V_{\mathcal{H}}^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q_{\mathcal{H}}^{\pi}(s, a)] - \underbrace{\mathbb{E}_{a \sim \pi(\cdot|s)} \log \pi(a|s)}_{\mathcal{H}(\pi(\cdot|s))}$$

Substituting this into the right hand side of the inequality above, we get:

$$-\mathbb{E}_{a \sim \pi_{\text{new}}(\cdot|s)} [Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) - \log \pi_{\text{new}}(a|s)] \leq -V_{\mathcal{H}}^{\pi_{\text{old}}}(s)$$

Using this upper bound on $V_{\mathcal{H}}^{\pi_{\text{old}}}(s')$, we upper bound $Q_{\mathcal{H}}^{\pi_{\text{old}}}$:

$$\begin{aligned} Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} [V_{\mathcal{H}}^{\pi_{\text{old}}}(s')] \\ &\leq \underbrace{r(s, a) - \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s, a) \\ a' \sim \pi_{\text{new}}(\cdot|s')}} \log \pi_{\text{new}}(a'|s')}_{r_{\mathcal{H}}^{\pi_{\text{new}}}(s, a)} + \gamma \mathbb{E}_{\substack{s' \sim P(\cdot|s, a) \\ a' \sim \pi_{\text{new}}(\cdot|s')}} [Q_{\mathcal{H}}^{\pi_{\text{old}}}(s', a')] \end{aligned}$$

Iterating this inequality:

$$Q_{\mathcal{H}}^{\pi_{\text{old}}}(s, a) \leq \mathbb{E}_{\pi_{\text{new}}} \left[\sum_{t=1}^{\infty} \gamma^{t-1} r_{\mathcal{H}}^{\pi_{\text{new}}}(s_t, a_t) | s_1 = s, a_1 = a \right] = Q_{\mathcal{H}}^{\pi_{\text{new}}}(s, a).$$

This inequality holds for all s, a , completing the proof. $\square$

Exercise 4. (Reparametrization trick) Let the loss function be defined as:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}} \left(\pi_{\theta}(\cdot|s) \left\| \frac{\exp(Q_{\mathcal{H},\phi}(s, \cdot))}{Z_{\phi}(s)} \right\| \right) \right],$$

and assume that there exists a function g_{θ} such that for $\varepsilon \sim \mathcal{N}(0, 1)$ and $a \sim \pi_{\theta}(\cdot|s)$, we have $a = g_{\theta}(\varepsilon, s)$. First, show that:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}, \varepsilon \sim \mathcal{N}(0,1)} \left[\log \pi_{\theta}(g_{\theta}(\varepsilon, s)|s) - Q_{\mathcal{H},\phi}(s, g_{\theta}(\varepsilon, s)) \right] + \text{const}$$

where const is with respect to θ . Then, derive $\nabla_{\theta} J(\theta)$. What functions must be differentiable for this method to work? What requirements are necessary for REINFORCE-like gradient estimation? What are the benefits of using this method?

Solution. Using the definition of the KL-divergence, we have:

$$\begin{aligned} J(\theta) &= \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot|s)} \log \frac{\pi_{\theta}(a|s)}{\frac{\exp(Q_{\mathcal{H},\phi}(s, a))}{Z_{\phi}(s)}} \\ &= \mathbb{E}_{s \sim \mathcal{D}, a \sim \pi_{\theta}(\cdot|s)} \left[\log \pi_{\theta}(a|s) - Q_{\mathcal{H},\phi}(s, a) \right] + \underbrace{\mathbb{E}_{s \sim \mathcal{D}} \log Z_{\phi}(s)}_{\text{const}} \end{aligned}$$

Next, applying reparameterization trick with $a = g_{\theta}(\varepsilon, s)$, we obtain:

$$J(\theta) = \mathbb{E}_{s \sim \mathcal{D}, \varepsilon \sim \mathcal{N}(0,1)} \left[\log \pi_{\theta}(g_{\theta}(\varepsilon, s)|s) - Q_{\mathcal{H},\phi}(s, g_{\theta}(\varepsilon, s)) \right] + \text{const}$$

To compute ∇_{θ} , note that s and ε are independent of θ , allowing the gradient to be brought inside the expectation:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s \sim \mathcal{D}, \varepsilon \sim \mathcal{N}(0,1)} \left[\nabla_{\theta} \log \pi_{\theta}(g_{\theta}(\varepsilon, s)|s) - \nabla_{\theta} Q_{\mathcal{H},\phi}(s, g_{\theta}(\varepsilon, s)) \right]$$

Using the chain rule, we calculate:

$$\nabla_{\theta} Q_{\mathcal{H},\phi}(s, g_{\theta}(\varepsilon, s)) = \frac{\partial g_{\theta}(\varepsilon, s)}{\partial \theta} \frac{\partial Q_{\mathcal{H},\phi}(s, a)}{\partial a} \Big|_{a=g_{\theta}(\varepsilon, s)}$$

Here, $\frac{\partial g_{\theta}(\varepsilon, s)}{\partial \theta}$ is a matrix of size $\dim(\theta) \times \dim(a)$, while $\frac{\partial Q_{\mathcal{H},\phi}(s, a)}{\partial a} \Big|_{a=g_{\theta}(\varepsilon, s)}$ is a vector of size $\dim(a)$. Similarly, for the gradient of $\log \pi_{\theta}$:

$$\nabla_{\theta} \log \pi_{\theta}(g_{\theta}(\varepsilon, s)|s) = \frac{\partial \log \pi_{\theta}(a|s)}{\partial \theta} \Big|_{a=g_{\theta}(\varepsilon, s)} + \frac{\partial g_{\theta}(\varepsilon, s)}{\partial \theta} \frac{\partial \log \pi_{\theta}(a|s)}{\partial a} \Big|_{a=g_{\theta}(\varepsilon, s)}$$

Combining these terms gives the final gradient expression.

For this method to work:

1. $\pi_{\theta}(a|s)$ must be differentiable with respect to θ and actions a ,
2. $g_{\theta}(\varepsilon, s)$ must be differentiable with respect to θ ,
3. $Q_{\mathcal{H},\phi}(s, a)$ must be differentiable with respect to a .

In contrast, REINFORCE-like gradient estimation only requires π_{θ} to be differentiable with respect to θ . However, in cases where Q is parameterized by a neural network (as with Q_{ϕ} here), all necessary derivatives can be computed efficiently using back-propagation. Benefits are both lower variance in gradient estimates, as well as more stable convergence as we can leverage back-propagation through π_{θ}, g_{θ} and Q_{ϕ} .

Exercise 5. Answer the following questions:

1. Is SAC an off- or on-policy algorithm? Why?
2. Where does SAC perform exploration?
3. Does this algorithm work for discrete/continuous action spaces?
4. Note that actions $\tilde{a}'$ are not sampled from the buffer but from the current policy. Why is this the case?
5. One possible parametrization of policy π_θ is using squashing function $\tilde{a}_\theta(s) = \tanh(\mu_\theta(s) + \sigma_\theta(s) \odot \varepsilon)$ with $\varepsilon \sim \mathcal{N}(0, I)$. Explain logic behind such parametrization.
6. What would happen if the target networks were updated as $\phi_{\text{target},\ell} \leftarrow \phi_\ell$ (with $\rho = 0$)?
7. Why are two target networks necessary in SAC?
8. What does the parameter α represent? Which algorithm is closest to SAC when $\alpha = 0$, and why?
9. Fill in the table for SAC. Do all the terms in the subtitle (Off-Policy Maximum Entropy Deep RL with a Stochastic Actor) make sense?

Solution. 1. SAC is off-policy because it uses transitions sampled from the replay buffer $\mathcal{D}$ to update targets.

2. SAC incorporates exploration during the policy update step by selecting a policy that maximizes entropy.

3. SAC can be applied to both discrete and continuous action spaces, provided gradients are handled appropriately.

4. Actions $\tilde{a}'_\theta$ must be sampled from the current policy to update it. However, the states s' , used to condition the policy for these sampled actions, are drawn from the replay buffer.

5. The policy is parameterized by μ_θ (mean) and σ_θ (standard deviation) for simplicity. The tanh function "squashes" the outputs to the range $[-1, 1]$, which is often suitable for practical applications requiring bounded actions.

6. Hard updates $\phi_{\text{target},\ell} \leftarrow \phi_\ell$ can cause instabilities. Soft updates (ex. with $\rho < 1$) gradually adjust the target network and help stabilize training.

7. SAC uses two target networks to mitigate maximization bias. Refer to the materials from Exercise Session 3 for more details.

8. The parameter α controls the strength of entropy regularization. A higher α encourages greater exploration by weighting the entropy term more heavily. When $\alpha = 0$, SAC behaves similarly to DDPG, as it removes entropy regularization entirely.

9. Summary for SAC:

- On/Off-Policy: SAC is off-policy due to its reliance on replay buffer sampling;
- Action Space: Supports both discrete and continuous action spaces;
- Sample Efficiency: High, due to off-policy updates;
- Exploration: Achieved via entropy-regularized policy updates;
- Stability: Moderately stable with soft updates and entropy regularization;
- Biggest disadvantage: Requires careful tuning of parameter α .